

利用 kubeadm 创建高可用集群

利用 kubeadm 创建高可用集群

本文讲述了使用 kubeadm 设置一个高可用的 Kubernetes 集群的两种不同方式：

- 使用具有堆叠的控制平面节点。这种方法所需基础设施较少。etcd 成员和控制平面节点位于同一位置。
- 使用外部 etcd 集群。这种方法所需基础设施较多。控制平面的节点和 etcd 成员是分开的。

在下一步之前，你应该仔细考虑哪种方法更好地满足你的应用程序和环境的需求。高可用拓扑选项 讲述了每种方法的优缺点。

如果你在安装 HA 集群时遇到问题，请在 kubeadm 问题跟踪里向我们提供反馈。

你也可以阅读升级文档。

这篇文档没有讲述在云提供商上运行集群的问题。在云环境中，此处记录的方法不适用于类型为 LoadBalancer 的服务对象，也不适用于具有动态 PersistentVolume 的对象。

根据集群控制平面所选择的拓扑结构不同，准备工作也有所差异：

需要准备：

- 配置满足 kubeadm 的最低要求 的三台机器作为控制面节点。控制平面节点为奇数有利于机器故障或者分区故障时重新选举。
 - 机器已经安装好，并正常运行
- 配置满足 kubeadm 的最低要求 的三台机器作为工作节点
 - 机器已经安装好，并正常运行
- 在集群中，确保所有计算机之间存在全网络连接（公网或私网）
- 在所有机器上具有 sudo 权限
 - 可以使用其他工具；本教程以 sudo 举例
- 从某台设备通过 SSH 访问系统中所有节点的能力
- 所有机器上已经安装 kubeadm 和 kubelet

拓扑详情请参考堆叠（Stacked）etcd 拓扑。

需要准备：

- 配置满足 kubeadm 的最低要求的三台机器作为控制面节点。控制平面节点为奇数有利于机器故障或者分区故障时重新选举。
 - 机器已经安装好，并正常运行
- 配置满足 kubeadm 的最低要求的三台机器作为工作节点
 - 机器已经安装好，并正常运行
- 在集群中，确保所有计算机之间存在全网络连接（公网或私网）
- 在所有机器上具有 sudo 权限
 - 可以使用其他工具；本教程以 sudo 举例
- 从某台设备通过 SSH 访问系统中所有节点的能力
- 所有机器上已经安装 kubeadm 和 kubelet

还需要准备：

- 给 etcd 集群使用的另外至少三台机器。为了分布式一致性算法达到更好的投票效果，集群必须由奇数个节点组成。
 - 机器上已经安装 kubeadm 和 kubelet。
 - 机器上同样需要安装好容器运行时，并能正常运行。

拓扑详情请参考外部 etcd 拓扑。

容器镜像 {#container-images}

每台主机需要能够从 Kubernetes 容器镜像仓库（registry.k8s.io）读取和拉取镜像。想要在无法拉取 Kubernetes 仓库镜像的机器上部署高可用集群也是可行的。通过其他的手段保证主机上已经有对应的容器镜像即可。

命令行 {#kubectl}

一旦集群创建成功，需要在 PC 上安装 kubectl 用于管理 Kubernetes。为了方便故障排查，也可以在每个控制平面节点上安装 kubectl。

这两种方法的第一步

为 kube-apiserver 创建负载均衡器

负载均衡器有很多种可能的配置方式。以下仅提供一个配置的例子。你的集群可能需要进行不同的配置。

1. 创建一个名为 kube-apiserver 的负载均衡器解析 DNS。

- 在云环境中，应该将控制平面节点放置在 TCP 转发负载均衡后面。该负载均衡器将流量分配给目标列表中所有运行状况良好的控制平面节点。API 服务器的健康检查是在 kube-apiserver 的监听端口（默认值 :6443）上进行的一个 TCP 检查。
- 不建议在云环境中直接使用 IP 地址。
- 负载均衡器必须能够在 API 服务器端口上与所有控制平面节点通信。它还必须允许其监听端口的入站流量。
- 确保负载均衡器的地址始终匹配 kubeadm 的 ControlPlaneEndpoint 地址。
- 阅读软件负载均衡选项指南 以获取更多详细信息。

2. 添加第一个控制平面节点到负载均衡器并测试连接：

```
nc -v LOAD_BALANCER_IP PORT
```

由于 API 服务器尚未运行，预期会出现一个连接拒绝错误。然而超时意味着负载均衡器不能和控制平面节点通信。如果发生超时，请重新配置负载均衡器与控制平面节点进行通信。

3. 将其余控制平面节点添加到负载均衡器目标组。

使用堆控制平面和 etcd 节点

控制平面节点的第一步

1. 初始化控制平面：

```
sudo kubeadm init --control-plane-endpoint  
"LOAD_BALANCER_DNS:LOAD_BALANCER_PORT" --upload-certs
```

- 你可以使用 --kubernetes-version 标志来设置要使用的 Kubernetes 版本。建议将 kubeadm、kebelet、kubectl 和 Kubernetes 的版本匹配。
- 这个 --control-plane-endpoint 标志应该被设置成负载均衡器的地址或 DNS 和端口。
- 这个 --upload-certs 标志用来将在所有控制平面实例之间的共享证书上传到集群。如果正好相反，你更喜欢手动地通过控制平面节点或者使用自动化工具复制证书，请删除此标志并参考如下部分证书分配手册。

在使用 `kubeadm init` 时，标记 `--config` 和 `--certificate-key` 不能混用，因此如果你要使用 `kubeadm` 配置，你必须在相应的配置结构（位于 `InitConfiguration` 和 `JoinConfiguration: controlPlane`）添加 `certificateKey` 字段。

一些 CNI 网络插件需要更多配置，例如指定 Pod IP CIDR，而其他插件则不需要。参考 CNI 网络文档。通过传递 `--pod-network-cidr` 标志添加 Pod CIDR，或者你可以使用 `kubeadm` 配置文件，在 `ClusterConfiguration` 的 `networking` 对象下设置 `podSubnet` 字段。

输出类似于：

...

You can now join any number of control-plane node by running the following command on each as a root:

```
kubeadm join 192.168.0.200:6443 --token 9vr73a.a8uxyaju799qwdjv --discovery-  
token-ca-cert-hash  
sha256:7c2e69131a36ae2a042a339b33381c6d0d43887e2de83720eff5359e26aec866  
--control-plane --certificate-key  
f8902e114ef118304e561c3ecd4d0b543adc226b7a07f675f56564185ffe0c07
```

Please note that the certificate-key gives access to cluster sensitive data, keep it secret!

As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can use `kubeadm init phase upload-certs` to reload certs afterward.

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 192.168.0.200:6443 --token 9vr73a.a8uxyaju799qwdjv --discovery-  
token-ca-cert-hash  
sha256:7c2e69131a36ae2a042a339b33381c6d0d43887e2de83720eff5359e26aec866
```

- 将此输出复制到文本文件。稍后你将需要它来将控制平面节点和工作节点加入集群。
- 当使用 `--upload-certs` 调用 `kubeadm init` 时，主控制平面的证书被加密并上传到 `kubeadm-certs Secret` 中。
- 要重新上传证书并生成新的解密密钥，请在已加入集群节点的控制平面上使用以下命令：

```
sudo kubeadm init phase upload-certs --upload-certs
```

- 你还可以在 init 期间指定自定义的 --certificate-key，以后可以由 join 使用。要生成这样的密钥，可以使用以下命令：

```
kubeadm certs certificate-key
```

证书密钥是一个十六进制编码的字符串，它是一个 32 字节大小的 AES 密钥。

kubeadm-certs Secret 和解密密钥会在两个小时后失效。

正如命令输出中所述，证书密钥可访问集群敏感数据。请妥善保管！

2. 应用你所选择的 CNI 插件：请遵循以下指示 安装 CNI 驱动。如果适用，请确保配置与 kubeadm 配置文件中指定的 Pod CIDR 相对应。

在进行下一步之前，必须选择并部署合适的网络插件。否则集群不会正常运行。

3. 输入以下内容，并查看控制平面组件的 Pod 启动：

```
kubectl get pod -n kube-system -w
```

其余控制平面节点的步骤

对于每个其他控制平面节点，你应该：

1. 执行先前由第一个节点上的 kubeadm init 输出提供给你的 join 命令。它看起来应该像这样：

```
sudo kubeadm join 192.168.0.200:6443 --token 9vr73a.a8uxyaju799qwdjv --  
discovery-token-ca-cert-hash  
sha256:7c2e69131a36ae2a042a339b33381c6d0d43887e2de83720eff5359e26aec866  
--control-plane --certificate-key  
f8902e114ef118304e561c3ecd4d0b543adc226b7a07f675f56564185ffe0c07
```

- 这个 --control-plane 标志通知 kubeadm join 创建一个新的控制平面。
- --certificate-key ... 将导致从集群中的 kubeadm-certs Secret 下载控制平面证书并使用给定的密钥进行解密。

由于集群节点通常按顺序初始化，CoreDNS Pod 很可能全部运行在第一个控制平面节点上。为了提供更高的可用性，请在加入至少一个新节点后，使用 `kubectl -n kube-system rollout restart deployment coredns` 重新平衡 CoreDNS Pod。

外部 etcd 节点

使用外部 etcd 节点设置集群类似于用于堆叠 etcd 的过程，不同之处在于你应该首先设置 etcd，并在 kubeadm 配置文件中传递 etcd 信息。

设置 etcd 集群

1. 按照这里的指示去设置。
2. 根据这里的描述配置 SSH。
3. 将以下文件从集群中的任一 etcd 节点复制到第一个控制平面节点：

```
scp /etc/kubernetes/pki/etcd/ca.crt "${CONTROL_PLANE}":  
scp /etc/kubernetes/pki/apiserver-etcd-client.crt "${CONTROL_PLANE}":  
scp /etc/kubernetes/pki/apiserver-etcd-client.key "${CONTROL_PLANE}":
```

- 用第一台控制平面节点的 user@host 替换 CONTROL_PLANE 的值。

设置第一个控制平面节点

1. 用以下内容创建一个名为 kubeadm-config.yaml 的文件：

```
---  
apiVersion: kubeadm.k8s.io/v1beta4  
kind: ClusterConfiguration  
kubernetesVersion: stable  
controlPlaneEndpoint: "LOAD_BALANCER_DNS:LOAD_BALANCER_PORT" # change  
this (see below)  
etcd:  
  external:  
    endpoints:  
      - https://ETCD_0_IP:2379 # 适当地更改 ETCD_0_IP  
      - https://ETCD_1_IP:2379 # 适当地更改 ETCD_1_IP  
      - https://ETCD_2_IP:2379 # 适当地更改 ETCD_2_IP  
    caFile: /etc/kubernetes/pki/etcd/ca.crt  
    certFile: /etc/kubernetes/pki/apiserver-etcd-client.crt  
    keyFile: /etc/kubernetes/pki/apiserver-etcd-client.key
```

这里的堆叠（stacked）etcd 和外部 etcd 之前的区别在于设置外部 etcd 需要一个 etcd 的 external 对象下带有 etcd 端点的配置文件。如果是内部 etcd，是自动管理的。

- 在你的集群中，将配置模板中的以下变量替换为适当值：

- LOAD_BALANCER_DNS
- LOAD_BALANCER_PORT
- ETCD_0_IP
- ETCD_1_IP
- ETCD_2_IP

以下的步骤与设置内置 etcd 的集群是相似的：

1. 在节点上运行 `sudo kubeadm init --config kubeadm-config.yaml --upload-certs` 命令。
2. 记下输出的 `join` 命令，这些命令将在以后使用。
3. 应用你选择的 CNI 插件。

在进行下一步之前，必须选择并部署合适的网络插件。否则集群不会正常运行。

其他控制平面节点的步骤

步骤与设置内置 etcd 相同：

- 确保第一个控制平面节点已完全初始化。
- 使用保存到文本文件的 `join` 命令将每个控制平面节点连接在一起。建议一次加入一个控制平面节点。
- 不要忘记默认情况下，`--certificate-key` 中的解密密钥会在两个小时后过期。

列举控制平面之后的常见任务

安装工作节点

你可以使用之前存储的 `kubeadm init` 命令的输出将工作节点加入集群中：

```
sudo kubeadm join 192.168.0.200:6443 --token 9vr73a.a8uxyaju799qwdjv --discovery-  
token-ca-cert-hash  
sha256:7c2e69131a36ae2a042a339b33381c6d0d43887e2de83720eff5359e26aec866
```

手动证书分发 {#manual-certs}

如果你选择不将 `kubeadm init` 与 `--upload-certs` 命令一起使用，则意味着你将必须手动将证书从主控制平面节点复制到将要加入的控制平面节点上。

有许多方法可以实现这种操作。下面的例子使用了 ssh 和 scp：

如果要在单独的一台计算机控制所有节点，则需要 SSH。

1. 在你的主设备上启用 ssh-agent，要求该设备能访问系统中的所有其他节点：

```
eval $(ssh-agent)
```

2. 将 SSH 身份添加到会话中：

```
ssh-add ~/.ssh/path_to_private_key
```

3. 检查节点间的 SSH 以确保连接是正常运行的

- SSH 到任何节点时，请确保添加 -A 标志。此标志允许你通过 SSH 登录到节点后从该节点上访问你自己 PC 上的 SSH 代理。如果你不完全信任该节点上的用户会话安全，可以考虑使用其他替代方法。

```
ssh -A 10.0.0.7
```

- 当在任何节点上使用 sudo 时，请确保保持环境变量设置，以便 SSH 转发能够正常工作：

```
sudo -E -s
```

4. 在所有节点上配置 SSH 之后，你应该在运行过 kubeadm init 命令的第一个控制平面节点上运行以下脚本。该脚本会将证书从第一个控制平面节点复制到另一个控制平面节点：

在以下示例中，用其他控制平面节点的 IP 地址替换 CONTROL_PLANE_IPS。

```
USER=ubuntu # 可定制
```

```
CONTROL_PLANE_IPS="10.0.0.7 10.0.0.8"
```

```
for host in ${CONTROL_PLANE_IPS}; do
```

```
  scp /etc/kubernetes/pki/ca.crt "${USER}"@$host:
```

```
  scp /etc/kubernetes/pki/ca.key "${USER}"@$host:
```

```
  scp /etc/kubernetes/pki/sa.key "${USER}"@$host:
```

```
  scp /etc/kubernetes/pki/sa.pub "${USER}"@$host:
```

```
  scp /etc/kubernetes/pki/front-proxy-ca.crt "${USER}"@$host:
```

```
  scp /etc/kubernetes/pki/front-proxy-ca.key "${USER}"@$host:
```

```
  scp /etc/kubernetes/pki/etcd/ca.crt "${USER}"@$host:etcd-ca.crt
```

```
  # 如果你正使用外部 etcd，忽略下一行
```



```
scp /etc/kubernetes/pki/etcd/ca.key "${USER}"@$host:etcd-ca.key  
done
```

只需要复制上面列表中的证书。kubeadm 将负责生成其余证书以及加入控制平面实例所需的 SAN。如果你错误地复制了所有证书，由于缺少所需的 SAN，创建其他节点可能会失败。

5. 然后，在每个即将加入集群的控制平面节点上，你必须先运行以下脚本，然后再运行 kubeadm join。该脚本会将先前复制的证书从主目录移动到 /etc/kubernetes/pki：

```
USER=ubuntu # 可定制  
mkdir -p /etc/kubernetes/pki/etcd  
mv /home/${USER}/ca.crt /etc/kubernetes/pki/  
mv /home/${USER}/ca.key /etc/kubernetes/pki/  
mv /home/${USER}/sa.pub /etc/kubernetes/pki/  
mv /home/${USER}/sa.key /etc/kubernetes/pki/  
mv /home/${USER}/front-proxy-ca.crt /etc/kubernetes/pki/  
mv /home/${USER}/front-proxy-ca.key /etc/kubernetes/pki/  
mv /home/${USER}/etcd-ca.crt /etc/kubernetes/pki/etcd/ca.crt  
# 如果你正使用外部 etcd，忽略下一行  
mv /home/${USER}/etcd-ca.key /etc/kubernetes/pki/etcd/ca.key
```